MEMS-based Disk Buffer for Streaming Multimedia

University of California, Santa Barbara

Abstract

The number of simultaneous continuous media streams that a single disk-drive can potentially support is directly proportional to the exponential increase in the sequential disk throughput. However, slower improvements in disk access times necessitate the use of large DRAM buffers to improve overall streaming throughput. The unique access and throughput characteristics of MEMS-based storage devices promise to bridge this gap. In this paper, we propose using MEMS-based storage devices as an additional layer between the disk and DRAM for buffering and caching real-time streaming data. An extensive analytical study shows that such an addition to the storage hierarchy can reduce the buffering cost and/or improve the throughput of streaming servers significantly.

1 Introduction

Economical delivery of continuous (or streaming) media-data requires high disk throughput and low memory use. However, due to the increasing gap between the improvements in disk bandwidth and disk access times (both seek time and rotational delay), achieving high disk throughput often requires a large DRAM buffer and hence increases system cost. Large DRAM buffer is especially necessary for servers which stream to a large number of clients. With the availability of high-speed Internet access at home, the number of users willing to use on-demand streaming multimedia is rapidly increasing. However, cost of systems that can support thousands of streams from a single box is still large, mainly caused by large buffering cost.

Emerging MEMS-based storage devices promise to bridge the performance gap between DRAM and disk drives. MEMS devices are predicted to be an order of magnitude cheaper than DRAM, while offering an order of magnitude faster access times than disk drives [6]. These devices offer a unique low-cost solution for the streaming applications. In this study, we propose an analytical framework to evaluate the effective use of MEMS in a media server. Specifically, we derive analytical models for studying two MEMS configurations, *using MEMS as a buffer* between DRAM and disk, and *using MEMS as a cache*.

- *MEMS as a buffer.* When acting as a buffer for speed-matching between the disk drive and DRAM, a MEMS-based storage provides superior access characteristics to DRAM, thus reducing the size of DRAM buffer. In addition, at a magnitude cheaper cost compared to DRAM, MEMS can provide larger buffer space for accessing the disk in large sequential blocks, resulting in high disk throughput.
- *MEMS as a cache.* When acting as a cache, the MEMS-based storage can replicate popular streams. Effective caching can reduce the DRAM buffer requirement and hence can improve the total streaming throughput.

At first glance, the issues involved in placing MEMS between DRAM and disk either as a buffer or as a cache, seem straightforward enough. However, the real-time IO requirement of the media data and the mismatch between the transfer rates of disk and DRAM make several MEMS configurations infeasible or counter-productive. Our extensive analytical and empirical studies reveal the following design principles:

1. MEMS devices, when used to buffer streaming data, must be used to buffer streams with low and medium bit-rate streams. Our study based on the projected MEMS, disk drive, and DRAM characteristics for the year 2007 suggest that buffering streams which have greater than 1MB/s bit-rate requirement simply do not prove cost-effective. On the other hand, buffering lower bit-rate streams can offer a large (as much as 75%) reduction in buffering cost.

2. When the streaming content has a non-uniform popularity distribution, MEMS-based disk caching can improve server throughput by more than 100%.

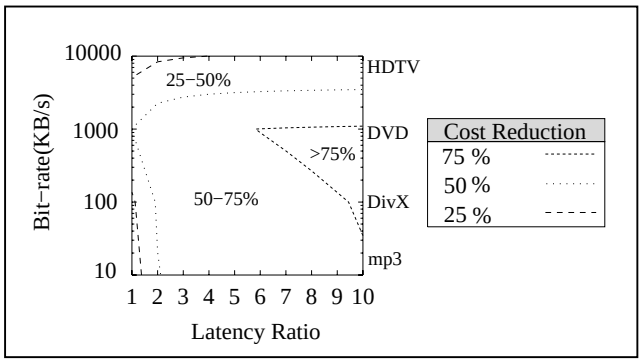


Figure 1: **The percentage reduction in buffering cost due to the addition of a MEMS-based disk buffer for a year 2007 streaming multimedia system**. The "FutureDisk" disk-drive predicted for the year 2007 has a sequential streaming bandwidth of up to 300MB/s. The X-axis depicts the ratio of latencies (projected to be around 5) of the disk-drive and the MEMS device. Two G3 [6] MEMS devices are used for buffering and deliver an aggregate throughput of 640MB/s. The low-entry cost of the MEMS device at $10 for a 10GB device makes these huge cost-reductions possible.

The rest of this paper is organized as follows: Section 2 briefly explains one possible architecture for the MEMS-based storage and presents the prediction for characteristics of DRAM, disk, and MEMS-based storage. Section 3 introduces the MEMS-based device as an intermediate buffer as well as a cache for streaming data. In Section 4 we derive a quantitative model for proposed system architecture. Section 5 presents analytical results which show the promise of significant cost reduction for systems that use an intermediate MEMS-based buffer. In Section 7, we make concluding remarks and suggest directions for future work.

2 MEMS-based Storage

Even after the long reign of Moore's Law, the basic memory hierarchy in computer systems has not changed significantly. At the non-volatile end, magnetic disks have managed to survive as the most cost-effective mass storage medium, and there are no alternative technologies which show promise for replacing them in the next decade [7]. Disk access times, however, are improving at the rate of only 10% per year. For more than a decade they have continued to lag behind the annual disk throughput increase of 40% and capacity increase of 60% [7]. Due to the increasing mismatch between disk access times and throughput, streaming multimedia systems require an increasing amount of buffering in the main system memory (DRAM) in order to utilize the available disk throughput.

MEMS-based storage is an emerging technology that may fill the performance gap between magnetic disks and DRAM [**?**]. Researchers at the Carnegie Melon University have proposed one possible architecture for a MEMS-based storage device [6, 3] depicted in Figure 2. They propose MEMS devices which would

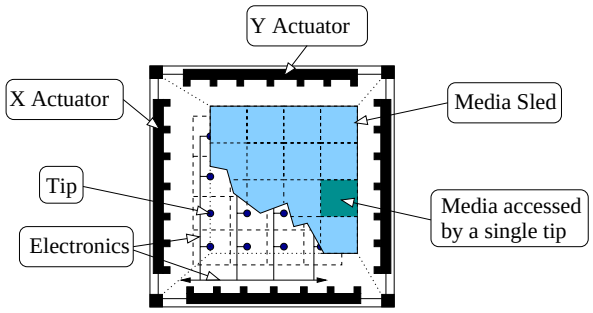


Figure 2: **MEMS-based storage architecture**.

be fabricated on-chip, but would use spring-mounted magnetic media sled as a storage medium. The media sled is placed above the two-dimensional array of micro-electro-mechanical read/write heads (tips). Actuators will move the media sled above the array of fixed tips along both the X and Y dimensions. Moving along the Y dimension at a constant velocity enables a subset (or all) of the tips to concurrently access data stored on the media sled. Using a large number of tips (of the order of thousands) concurrently, such a device can deliver high data throughput. The light-weight media sled of the MEMS device can be moved and positioned much faster than bulkier disk servo-mechanisms, thus cutting down access times by an order of magnitude. Additionaly, MEMS access time does not include rotational delay.

Table 1 summarizes important characteristics of different storage media for year 2002 and predicted values for year 2007 using trend studies. The MEMS device projections are borrowed from [6]; the disk drive projections are based on a study by IBM researchers [7]; and [**?**] provides the basis for DRAM predictions.

Year		DRAM	Disk	Mems
2002	Capacity [GB]	0.5	100	n/a
	Access time [ms]	0.05	$1-11$	n/a
	Bandwidth [MB/s]	$2,000$	$30-55$	n/a
	Cost/GB	\$200	\$2	n/a
	Cost/device	\$50- \$200	\$100- \$300	n/a
2007	Capacity [GB]	5	$1,000$	10
	Access time [ms]	0.03	$0.75-7$	$0.4-1$
	Bandwidth [MB/s]	$10,000$	$170-295$	300
	Cost/GB	\$20	\$0.2	\$1
	Cost/device	\$50- \$200	\$100- \$300	\$10

Table 1: **Storage media characteristics**.

Typically, all three storage mediums described in this section are well optimized for sequential access. The maximum DRAM throughput is achieved when data are accessed in sequential chunks, about the size of the largest cache block, in the main memory hierarchy. These are typically tens to hundreds of bytes. However, both magnetic disks and MEMS-based storage devices (MEMS) have much larger access times than DRAM. These devices have to be accessed in much larger chunks to mask the access overheads, the MEMS device being the faster of the two by an order of magnitude. Figure 3 shows the effective throughput of the disk drive and the MEMS device depending on the average size of sequential IO requests to these devices. In Figure 3 we use the maximum access times for servicing MEMS IO requests, and the average access times for disk IO requests.

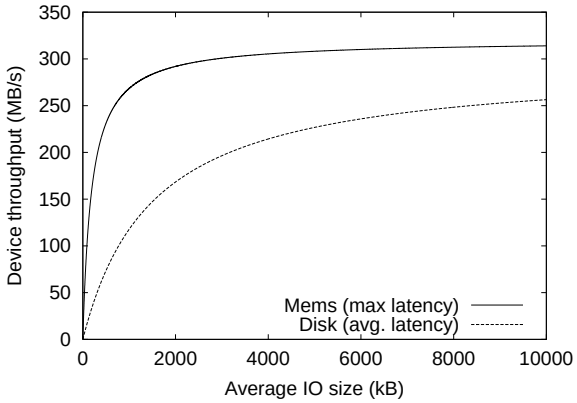


Figure 3: **Effective throughput for the FutureDisk and G3 MEMS devices**. A larger IO size delivers a greater device throughput.

3 MEMS-based Streaming

To service streaming data from the secondary disk-drive storage, there exists a tradeoff between the achieved disk utilization and the amount of buffered stream data. Operating the disk drive at a higher utilization requires accessing the disk using larger IOs and consequently requiring more data buffering for the individual streams. In traditional multimedia systems, the buffering requirement is addressed using the system memory (DRAM). MEMS-based storage devices can be used to offload part of the DRAM buffering requirement. They can also be used as caches to provide faster access to multimedia content. Before we investigate these possibilities, we must first briefly revisit the scheduling of disk IO for streaming multimedia.

Due to the large sizes of streaming media data, they usually are stored on the disk drive. On request, chunks of the media stream are retrieved periodically into the system DRAM for consumption till the stream playback ends. To service real-time disk IOs for continuous media, several classes of disk scheduling algorithms have been proposed. The most popular among them are the *Quality Proportion Multi-Subscriber Servicing (QPMS)* or *Time-cycle* based scheduling [5] and the *Earliest Deadline First (EDF)* [2] scheduling approaches. In the rest of this paper, we shall adopt the time-cycle based scheduling model [5] for scheduling continuous media streams due to its simplicity[1]. In this model, time is split into basic units called IO cycles. In each IO cycle, the disk performs exactly one IO operation for each media stream.

IO cycle time: Given the stream bit-rates, the *IO cycle time* is the amount of time required to transfer a sufficient amount of data for each stream so as to sustain jitter-free playback.

The IO cycle time depends on the system configuration, the number and type of streams requested, and the disk scheduling algorithm employed. Two representative algorithms which make different trade-offs between the disk utilization and the buffering requirement are the *round-robin* and the *elevator* algorithms.

Round-robin scheduling: The scheduler services the streams in the same order in each time-cycle. Careful management of data buffers and precise scheduling [1] can reduce the total amount of buffering required to the amount of data read in one time-cycle. However, the disk does not optimize access to data depending on the data location and might suffer the worst-case access overhead for each I/O operation. The disk utilization is thus underestimated.

Elevator scheduling: In each time-cycle, the disk schedules IOs for streams in the order in which they are placed on the disk medium when the disk arm sweeps from the innermost cylinder to the outermost

[1] Of the two classes of continuous media schedulers, the *time-cycle* based scheduler has gained popularity due to its simplicity and inherent support for timing guarantees for individual streams. Apart from this, the time-cycle service model also simplifies admission control and the incorporation of starvation-free service for non-real-time data.

cylinder (or vice-versa). The scheduler thus bounds the total seek overhead for all disk accesses within each cycle to the full-stroke of the disk arm. However, since the scheduling order for streams may vary between cycles, the size of the data buffer is equal to twice the amount of data read in one time-cycle. Thus, disk utilization is increased at the cost of increased buffering requirement.

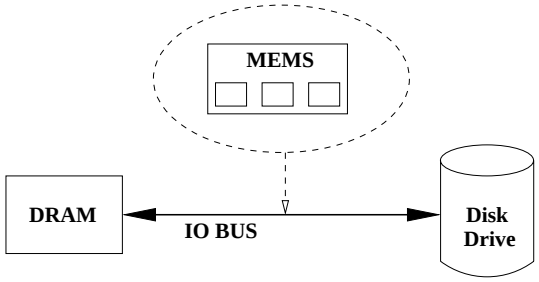


Figure 4: **The MEMS Architecture**

We now present the architecture for the MEMS-based disk buffer. Figure 4 depicts the new system architecture that includes the MEMS device in the storage hierarchy. The MEMS storage module can consist of multiple MEMS devices to provide greater storage capacity and throughput. The MEMS device is accessed over the IO bus. It can be envisioned as being part of the disk drive or as an independent device. In either case, IOs can be scheduled on the MEMS device as well as the disk drive independently. Similar to disk caches found on current-day disk drives, the MEMS storage device also consists of a cache for buffering incoming or outgoing data. In what follows, we present two possible scenarios in which such an architecture can be used improve the performance of a multimedia system.

3.1 MEMS Multimedia Buffer

MEMS-based storage devices are predicted to be at least an order of magnitude cheaper than DRAM storage [6]. Using a MEMS storage device as an intermediate buffering medium reduces the cost of buffering multimedia data, thus enabling the disk drive to be better utilized at the same cost. However, since the MEMS device does not offer uniform memory access, the DRAM buffering used by the operating system for fast random access to data cannot be eliminated completely. The MEMS device does offers lower access latencies than disk drives do, thus enabling the use of smaller IOs to achieve disk utilization levels. The smaller IOs on the MEMS device imply a smaller DRAM buffering requirement. The MEMS device can thus act as a speed-matching buffer between the disk drive and the system memory, in effect addressing the disk utilization and data buffering tradeoff.

Using MEMS-based storage as a speed-matching buffer implies that the MEMS-based device must handle both disk and DRAM data traffic simultaneously. To understand the service model, let us assume that the multimedia streams being serviced are all read streams, so that stream data read from the disk drive is buffered temporarily at the MEMS device before it is read into the DRAM. This model can be easily extended to address write streams.

To service buffered data from the MEMS device, we use the *IO time-cycle* service model proposed before for disk drives. Data is retrieved in cycles into the DRAM such that no individual stream experiences data underflow at the DRAM at any instant. At the same time, the data read from the disk drive must be written to the MEMS device. The disk IO scheduler controls the read operations at the disk drive. The MEMS IO scheduler controls the write operations for data read from the disk as well as read operations into the DRAM. In the steady state, the amount of data being written to the MEMS device is equal to the amount read from it. The MEMS IO bandwidth is thus split equally among read and write operations. Thus, although the MEMS device can help improve disk utilization, we must realize that to do so, it must

operate at twice the throughput as that of the disk drive. In order to minimize buffering requirements between the disk drive and the MEMS storage, the disk scheduler and the MEMS scheduler must therefore cooperate.

The MEMS buffer could comprise of multiple physical MEMS devices. As we shall see in the Section 5, using a bank of MEMS devices is may be required to buffer IOs for a single disk. The low entry-cost of these devices makes such configurations practical. First, we present a feasible IO schedule that maintains real-time guarantees, when a single MEMS device is used for buffering. We then extend our methodology to work with a bank of MEMS devices.

3.1.1 IO Scheduling: Single MEMS

We now present one possible IO schedule which guarantees real-time data retrieval from the disk on a single MEMS device. As noted earlier, the MEMS device bandwidth must be twice that of the disk drive.

The MEMS IO scheduler services IOs on the MEMS device in rounds or *IO cycles*. In each IO cycle, the MEMS device services exactly one DRAM transfer for each of the N streams serviced by the system. The amount of data read for each stream is sufficient to sustain playback before the next IO for the stream is performed.

Apart from performing IOs for DRAM transfers, the MEMS device also services disk transfers within the IO cycle. However, not all streams are serviced disk transfers in each IO cycle at the MEMS device. The number of disk transfers serviced in each IO cycle depends on the system configuration. We explore this aspect further during our analysis in Section 4.

Figure 5 describes the IO and data transfer operations occurring during a single MEMS IO cycle. The X-axis is the time axis and the three horizontal lines depict the activity at the disk head, the MEMS head, and the DRAM, respectively. In this example, the system services 10 streams (i.e., $N = 10$.) The lightly shaded regions depict data which are read from the disk drive into the MEMS device. The dark regions depict data transferred between the MEMS device and the DRAM. The activity at the MEMS head shows the IO operations during a MEMS IO cycle. The MEMS device performs N small IO transfers from(to) the DRAM, and a few large disk transfers.

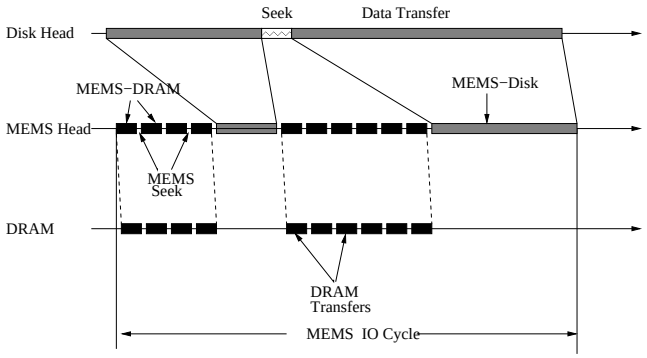


Figure 5: **MEMS IO Scheduling**. Activity at the MEMS head shows the set of IO operations being performed by the MEMS device in the duration of a MEMS IO cycle.

3.1.2 IO Scheduling: Multiple MEMS

According to certain predictions [8, 6], a single MEMS device might not be able to support twice the bandwidth of future disk drives. In such cases, buffering bandwidth must be increased by using a bank of k MEMS devices to provide a higher aggregate bandwidth required to utilize the disk to the fullest.

Using k MEMS devices in place of a single device, raises some interesting questions. *How should stream data be split across these devices? What constitutes the IO cycle in the MEMS buffer? To what use can any spare storage or bandwidth available on the MEMS devices be put to?* We answer each question in turn.

Placing stream data: Data for the streams served by the multimedia server can be placed in one of two ways:

- Stripe the buffered data for each stream across the MEMS bank.
- Buffer each stream on a single MEMS device.

Striping data for each stream across the k MEMS devices can be accomplished by splitting each disk IO into k parts and routing each part to a different MEMS device. The size of disk-side IOs performed on the MEMS device is reduced by a factor of k. In Section 2, we noted that a larger average IO size on the MEMS device increases its utilization. Striping can thus reduce reduce the achieved MEMS throughput.

Instead of striping each stream across the k MEMS devices, the set of streams could be split across the MEMS bank. Each stream is thus buffered on a single MEMS device. This would preserve the size of disk-side IO transfers. To achieve such a split, the disk IOs are routed to the MEMS devices in a round-robin fashion. Every kth disk IO is routed to the same MEMS device[2]. Routing each IO to a single MEMS device improves MEMS device utilization and is thus preferable to striping each disk IO across the MEMS bank.

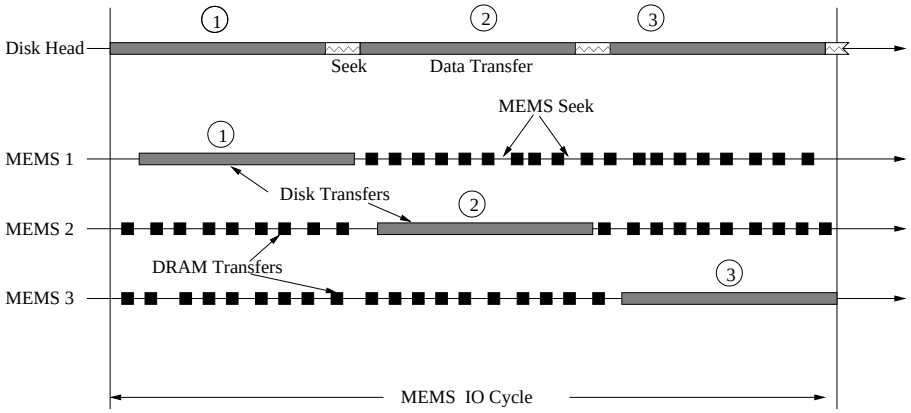


Figure 6: **IO Scheduling for multiple MEMS devices**. Activity at each MEMS head shows the set of IO operations being performed by the MEMS device in the duration of a MEMS IO cycle.

IO Cycle for k-device MEMS bank: Routing each disk IO to a single MEMS device splits the set of streams across the k MEMS devices in the bank. The notion of *IO cycle* for the MEMS bank can be defined in the same as that for a single device MEMS buffer. It is the time required to perform exactly one DRAM transfer for each of the N streams served by the system. For the sake of simplicity, let us assume that each MEMS device services $\frac{N}{k}$ streams. Figure 6 depicts the operations during an IO cycle at each MEMS device. The number of streams, N, is 57 and the number of MEMS devices in the bank is $k = 3$. For each disk IO, 19 DRAM transfers take place. The amount of data read from and written into the MEMS device is the same in the steady state[3].

[2]One might be concerned that this method requires that the number of streams served be divisible by k. However, this restriction can be circumvented by relaxing the requirement that each stream must be buffered always on the same device. As long as each disk IO is routed to a single MEMS device, this scheme is operable.

[3]It might appear that the IO bus bandwidth required for DRAM would be greater since they occur from multiple MEMS devices simultaneously. However, these transfers are easily time-shared and only consume as much bus-bandwidth as the effective throughput of the disk-drive.

Using the spare MEMS storage and bandwidth: Depending on the number and type of streams served and the capacity of the MEMS device bank, spare storage and/or bandwidth might be available at the MEMS device. If additional storage is available at the MEMS device, the operating system could use this storage for other non-real-time data, either as a persistent write buffer or as a cache for read data with temporal or spatial locality or as a disk prefetch buffer. The MEMS storage can also be used to cache entire streams as we shall explore next. Spare bandwidth, if available on the MEMS device, can be used for non-real-time traffic.

3.2 MEMS Multimedia Cache

Multimedia content usually has a well-defined popularity distribution and some content are accessed more frequently than others. Besides using MEMS storage as a buffer for streaming multimedia data from the disk drive, we can also use a MEMS storage device as a cache for popular multimedia content. Since the MEMS device offers low latency data access at throughput levels similar to those of the disk-drive, storing popular content on a MEMS cache can reduce the buffering requirement for streaming data and hence DRAM cost. By using k MEMS devices, we can also use the aggregate bandwidth of the MEMS cache for improving the server throughput.

One significant difference between using a MEMS device as a cache and as a buffer is that with caching, the MEMS cache behaves mostly as a read-only device. The MEMS cache is only updated to account for changes in stream popularity. This can be accomplished off-line, during service down-time.

To service streams from the cache, we use the IO time-cycle model similar to that proposed for disk-drives. However, there now exist two distinct IO cycles, one for the streams served from the disk-drive and the other for those served from the MEMS cache. The performance of either device depends on the available DRAM buffering and the number of streams served from it.

When more than one MEMS device is used for caching, the performance of the MEMS cache depends on the data management policy used for the MEMS bank. With multiple devices, we must ensure that the load on the MEMS devices is balanced. In this regard, we can draw on research from data management policies for disk arrays. We now investigate two cache-management policies, representing two classes of load balancing strategies, which ensure total load-balance across the MEMS bank. These approaches make different trade-offs to optimize for a sub-set of system configurations. More sophisticated load-balancing strategies, including hybrid approaches of the above, have been proposed in literature **??**. We investigate two simple, representative approaches as a first step.

3.2.1 Striped Cache-management

Using *striped cache-management*, each stream is bit or byte striped across all the k MEMS devices. There is no redundancy and data for each stream is simply distributed round-robin over the MEMS devices. To perform an IO on the MEMS cache, all the devices access exactly the same relative data location, in a lock-step fashion. The MEMS cache enjoys perfect load-balancing. The effective data transfer rate of the MEMS cache is k times that of a single device. The effective access latency of the MEMS cache is the same as that while accessing a single device. If N_m streams are served from the MEMS cache, the total number of seek operations in an IO cycle is $k \cdot N_m$. Using striped cache-management, perfect load-balancing is achieved at the cost of reduced access parallelism of the devices. In Section 4, we analyze this trade-off further.

3.2.2 Replicated Cache-management

Using *replicated cache-management*, the cached streams are replicated on the k MEMS devices. All the devices store exactly the same content. To perform an IO on the MEMS cache, any of the k devices

can be accessed. If the number of streams served from the MEMS cache is N_m, then each MEMS device services exactly $\frac{N_m}{k}$ of the streams. Since each MEMS device stores all the cached streams, such a division is possible. To service IOs for streams, each MEMS device operates independently servicing IO for its set of $\frac{N_m}{k}$ streams in rounds. The effective data transfer rate of the MEMS cache is k times that of a single device. Operating the k devices independently improves parallelism of access. The total number of seek operations in an IO cycle is only N as opposed to $k \cdot N_m$ in striped cache-management. Using replicated cache-management, perfect load balancing is achieved at the cost of reduced cache size due to data redundancy.

In Section 4, we further analyze the trade-offs involved in each policy.

4 Quantitative Analysis

In this section we derive quantitative models to analyze buffering requirements for systems supporting real-time streaming applications. We discuss two system configurations with MEMS-based storage device between the disk and DRAM:

- MEMS as a buffer between disk and DRAM. All data retrieved from disk to DRAM are first retrieved into MEMS and then transferred into DRAM.
- MEMS as a cache between disk and DRAM. Selected data are cached in MEMS. Requested data can be serviced from either disk or MEMS.

Parameter	Description
N	Number of continuous media streams
$\bar{B}$	Average bit-rate of the streams serviced $[B/s]$
k	Number of MEMS devices in system
R_{disk}	Sequential data transfer rate from disk media $[B/s]$
R_{mems}	Sequential data transfer rate from MEMS media $[B/s]$
$\bar{L}_{disk}$	Average latency of for disk IO operations $[s]$
$\bar{L}_{mems}$	Average latency of for MEMS IO operations $[s]$
C_{dram}	Unit DRAM cost $[\$/B]$
C_{mems}	Unit MEMS cost $[\$/B]$
$Size_{mems}$	MEMS capacity per device $[B]$
$Size_{disk}$	Disk capacity $[B]$
$S_{disk-dram}$	Average IO size from disk to DRAM $[B]$
$S_{disk-mems}$	Average IO size from disk to MEMS $[B]$
$S_{mems-dram}$	Average IO size from MEMS to DRAM $[B]$
T_{disk}	Disk IO cycle $[s]$
T_{mems}	MEMS IO cycle $[s]$

Table 2: Parameter definitions.

To evaluate MEMS' effectiveness as a buffer or as a cache, we need to compare the system cost with and without MEMS, for which we derive quantitative models. Let C_{dram} and C_{mems} denote the unit cost (\$/B) of DRAM and MEMS buffer, respectively. Furthermore, since we can only have whole number of MEMS devices in our system (k), we use per-device cost model for MEMS storage. These k MEMS devices cost $k \times C_{mems} \times Size_{mems}$ even if the system does not utilize all available MEMS storage.

4.1 MEMS as a Buffer

Let $S_{disk-drams}$, $S_{disk-mems}$, and $S_{mems-dram}$ denote the per-stream IO size from disk to DRAM, from disk to MEMS, and from MEMS to DRAM. Let k denote the number of MEMS devices in the system.

Let N denote the number of streams in the system. We can characterize the buffer cost as follows:
Buffer cost in the system without MEMS.

$$COST_{without_mems} = N \times C_{dram} \times S_{disk-dram}. \quad (1)$$

Buffer cost in the system with MEMS.

$$COST_{with_mems} = k \times C_{mems} \times Size_{mems} + N \times C_{dram} \times S_{mems-dram} \ , \quad where \ \ k \times Size_{mems} \geq N \times S_{disk-mems}. \quad (2)$$

Using MEMS devices in a streaming system is cost effective only if $COST_{with_mems} < COST_{without_mems}$.

In order to calculate system cost we need to first calculate IO sizes that guarantee the real-time streaming requirements. We next describe procedures to compute disk and MEMS IO sizes given the following four input parameters:

- N. The number of streams that the server supports.
- $\bar{B}$. The average bitrate of the N streams.
- R_d. The data transfer rate of device d. R_d is substituted by R_{disk} (disk transfer rate) or R_{mems} (MEMS transfer rate) depending on where the IO takes place.
- $\bar{L}_d$. The average latency of device d in a time-cycle. $\bar{L}_d$ is substituted by $\bar{L}_{disk}$ (disk latency) or $\bar{L}_{mems}$ (MEMS transfer rate) depending on the where the IO takes place. L_d also depends on the scheduling policy that is employed to manage device d.

In computing IO size, we make two assumptions that are commonly used in modeling a media server. First, we use the notion of *IO cycles* (Section 3). In each IO cycle, given the consumption rates of the individual streams, sufficient data is retrieved for each stream of the N streams so that the data buffers do not underflow to cause playback hiccups. Second, to simplify the analytical model, we assume all streams are in constant bit-rate (CBR)[4]. To assist the reader, Table 2 summarizes the parameters used in this paper.

Theorem 1. The minimum size of per-stream DRAM buffer required to satisfy real-time streaming requirements in a system which streams directly from the disk to DRAM is

$$S_{disk-dram} = \frac{N \times \bar{L}_{disk} \times R_{disk} \times \bar{B}}{R_{disk} - N \times \bar{B}} \ , \quad where \ \ R_{disk} > N \times \bar{B}. \quad (3)$$

Proof: Given N streams to support, in each IO cycle the disk must perform N IO operations, each consisting of a latency component and a data transfer component. Let T_{disk} denote the disk cycle time. Let L_{disk_i} denote the disk latency to start IO transfer for stream i. Then T_{disk} can be written as

$$T_{disk} \geq \sum_{i=1}^{N} L_{disk_i} + \frac{N \times T_{disk} \times \bar{B}}{R_{disk}}.$$

To fulfill the real-time data requirement, the amount of data transferred during T_{disk} must be sufficient to sustain the next T_{disk} time of playback for N streams. Thus the average disk IO size $S_{disk-dram} = T_{disk} \times \bar{B}$. Now, we isolate T_{disk} on the left-hand side of the equation, and rewrite T_{disk} as

$$T_{disk} \geq \frac{N \times \bar{L}_{disk} \times R_{disk}}{R_{disk} - N \times \bar{B}}.$$

[4]Many studies have shown that VBR can be modeled by CBR plus some memory cushion for handling bitrate variability.

Finally, given T_{disk} and $\bar{B}$, we can compute the average per-stream disk IO size as $S_{disk-dram} = T_{disk} \times \bar{B}$. □

Corollary 1. The minimum size of per-stream DRAM buffer required to satisfy real-time streaming requirements in a system which streams directly from the MEMS device to DRAM is

$$S_{mems-dram} = \frac{N \times \bar{L}_{mems} \times R_{mems} \times \bar{B}}{R_{mems} - N \times \bar{B}}, \quad where \quad R_{disk} > N \times \bar{B}. \tag{4}$$

Calculating IO sizes in a system that uses MEMS as an intermediate buffer between the disk and DRAM is more complex because we must consider the real-time requirements between the disk and MEMS as well as between the MEMS and DRAM.

Theorem 2. Minimum size of per-stream DRAM buffer in order to satisfy real-time streaming requirements in a system which uses k MEMS devices as a buffer is

$$S_{mems-dram} = \bar{B} \times \frac{C \times (1 + \frac{2k-2}{N}) \times T_{disk}}{T_{disk} - C}, \ where \ C = \frac{N \times \bar{L}_{mems} \times R_{mems}}{k \times R_{mems} - 2 \times (N + k - 1) \times \bar{B}}, \tag{5}$$

where T_{disk} is the largest value such that the following three conditions (real-time requirement, storage requirement, and scheduling requirement) are satisfied:

$$T_{disk} \geq \frac{N \times \bar{L}_{disk} \times R_{disk}}{R_{disk} - N \times \bar{B}} \quad , \quad where \quad R_{disk} > N \times \bar{B}; \tag{6}$$

$$2 \times N \times T_{disk} \times \bar{B} \leq k \times Size_{mems}; \tag{7}$$

$$\frac{T_{mems}}{T_{disk}} = \frac{M}{N}, \quad M < N, \quad M \in Integer. \tag{8}$$

Proof: Let M denote the number of disk IOs performed in one MEMS time-cycle (to simplify scheduling, we will let M be an integer). Finally, let k denote the number of MEMS devices acting as a buffer. When MEMS acts as a buffer, the data transfered from the disk into MEMS and from MEMS into DRAM must be equal. Consequently:

$$\begin{aligned} N \times S_{mems-dram} &= M \times S_{disk-mems} \\ \frac{N}{M} &= \frac{S_{disk-mems}}{S_{mems-dram}} = \frac{T_{disk}}{T_{mems}} \\ M &= N \times \frac{T_{mems}}{T_{disk}}, \quad \{N, M\} \in Integer. \end{aligned} \tag{9}$$

Equations 6 and 7 give a possible range for T_{disk}. However, the lower bound for T_{mems} has to be derived. Let's first express T_{mems} in a system with k MEMS devices so that real-time streaming requirements are satisfied:

$$T_{mems} \geq \left(\left\lceil \frac{N}{k} \right\rceil + \left\lceil \frac{M}{k} \right\rceil \right) \times \bar{L}_{mems} + (2 \times \left\lceil \frac{N}{k} \right\rceil \times \bar{B} \times T_{mems}) / R_{mems}. \tag{10}$$

Using Equation 10 we derive minimum value for T_{mems} as

$$\begin{aligned} T_{mems} \times \left(R_{mems} - 2 \times \left\lceil \frac{N}{k} \right\rceil \times \bar{B} \right) &\geq \left(\left\lceil \frac{N}{k} \right\rceil + \left\lceil \frac{M}{k} \right\rceil \right) \times \bar{L}_{mems} \times R_{mems} \\ T_{mems} &\geq \frac{(N + 2k - 2 + M) \times \frac{\bar{L}_{mems}}{k} \times k \times R_{mems}}{k \times R_{mems} - 2 \times (N + k - 1) \times \bar{B}}. \end{aligned} \tag{11}$$

Corollary 2. When N and M are divisible by k (or relatively large compared to k), k MEMS devices behave as a single MEMS device with both k times smaller average latency and k times larger throughput.

Let us now express minimum T_{mems} as a function of T_{disk} using Equations 6, 9, and 11 as

$$T_{mems} = \frac{C \times (1 + \frac{2k-2}{N}) \times T_{disk}}{T_{disk} - C} \ , \ where \ C = \frac{N \times \bar{L}_{mems} \times R_{mems}}{k \times R_{mems} - 2 \times (N + k - 1) \times \bar{B}}. \quad (12)$$

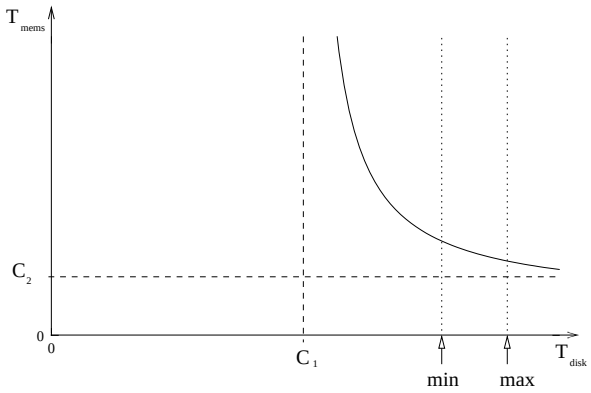


Figure 7: T_{mems} as a function of T_{disk}.

Figure 7 depicts the possible values of T_{mems} as a function of T_{disk} ($C_1 = C$ and $C_2 = C \times (1 + (2k-2)/N)$). Equation 6 gives minimum value for T_{disk} so that system satisfies real-time requirements. Maximum T_{disk} is bounded by the capacity of MEMS storage as in Equation 7. □

We can now find a minimum cost for streaming buffer by performing iterative method depicted in Figure 8.

Procedure FindMinimumBufferCost.
Input: N, $\bar{B}$, $\bar{L}_{disk}$, R_{disk}, $\bar{L}_{mems}$, R_{mems}.
Output: k, T_{mems}, and T_{disk}.

1. $MinCost = C_{dram} \times N \times S_{disk-dram}$
2. $k = 0$
3. **do**
4. $\quad k = k + 1$
5. $\quad$ Find T_{mems} using Theorem 2.
6. $\quad Cost = k \times C_{mems} \times Size_{mems} + C_{dram} \times N \times T_{mems} \times \bar{B}$
7. $\quad$ **if** $Cost < MinCost$ **then** $MinCost = Cost$
8. **while** $MinCost > (k + 1) \times C_{mems} \times Size_{mems}$

Figure 8: Procedure for finding minimum buffer cost.

4.2 MEMS as a Cache

If the multimedia content has a non-uniform popularity distribution, the MEMS storage can be used as a cache for popular streams. At first glance, this idea seems straightforward. However, streaming multimedia data are large in size and do not have temporal locality, and the only benefit of caching is the reduction in required DRAM buffering needed to satisfy real-time requirements.

Let h denote the hit-rate for the MEMS cache. Given N streams to service, $N \times h$ of them can thus be serviced from MEMS, and $N \times (1 - h)$ streams are serviced from disk. Similarly to the cost model for

MEMS multimedia buffer, we can characterize the cost of DRAM buffer and MEMS cache as follows:

$$\begin{aligned} COST_{with_mems-cache} &= k \times C_{mems} \times Size_{mems} + h \times N \times C_{dram} \times S_{mems-dram} \\ &\quad + (1-h) \times N \times C_{dram} \times S_{disk-dram}. \end{aligned} \tag{13}$$

Using Theorem 1 we can calculate $S_{disk-dram}$ as

$$S_{disk-dram} = \frac{(1-h) \times N \times \bar{L}_{disk} \times R_{disk} \times \bar{B}}{R_{disk} - (1-h) \times N \times \bar{B}} \quad , \quad where \quad R_{disk} > (1-h) \times N \times \bar{B}. \tag{14}$$

In order to calculate h and $S_{mems-dram}$, let us assume that the popularity distribution of content is specified by $X : Y$, where $X\%$ of the streams are accessed $Y\%$ of the time. Let us assume that both popular ($X\%$) and non-popular streams ($100\% - X$) are accessed uniformly in their class. This distribution is used so that we can estimate when is MEMS cache useful because streaming data do not have temporal locality and hence cache can be efficiently used only when the most of the popular streams can fit in MEMS. The capacity of the disk, $Size_{disk}$, is the total storage required for all the streams serviced by the system. Let the capacity of a single MEMS device be denoted by $Size_{mems}$. Let the percentage of movies cached is denoted by p. Then, the cache hit ratio, h, is

$$h = \begin{cases} \frac{p}{X} \times \frac{Y}{100\%} & \text{if } X \geq p, \\ \frac{Y}{100\%} + \frac{p-X}{100\%-X} \times \frac{100\%-Y}{100\%} & \text{otherwise.} \end{cases} \tag{15}$$

However, both p and $S_{mems-dram}$ depend on the cache management policy used for accessing MEMS cache. We explore two possible cases, namely replicated and striped multimedia cache (see Section 3.2).

4.2.1 Replicated Cache Management

Theorem 3. The minimum size of per-stream DRAM buffer required to satisfy real-time streaming requirements in a system which streams n streams from an array of k MEMS devices with replicated cache management is

$$S_{mems-dram} = \frac{(n+k-1) \times \frac{\bar{L}_{mems}}{k} \times (k \times R_{mems}) \times \bar{B}}{(k \times R_{mems}) - (n+k-1) \times \bar{B}} \quad , \quad where \quad k \times R_{mems} > (n+k-1) \times \bar{B}. \tag{16}$$

Proof: In replicated case, each MEMS device contains all cached streams. This simplifies streaming scheduler, and we can express MEMS time-cycle as

$$T_{mems} \geq \left\lceil \frac{n}{k} \right\rceil \times \bar{L}_{mems} + \left(\left\lceil \frac{n}{k} \right\rceil \times \bar{B} \times T_{mems} \right) / R_{mems}. \tag{17}$$

Similarly to Theorem 2, using Equation 17, we can derive Equation 16. □

Corollary 3. When N is divisible by k (or relatively large compared to k), k replicated cache behave as a single MEMS device with both k times smaller average latency and k times larger throughput.

For replicated case, percentage of cached streams is

$$p = \frac{Size_{mems}}{Size_{disk}}, \quad for\ any\ k.$$

Using Equation 15 we can calculate h and then use Theorem 3 to find per-stream buffer requirement:

$$S_{mems-dram} = \frac{h \times (N+k-1) \times \frac{\bar{L}_{mems}}{k} \times (k \times R_{mems}) \times \bar{B}}{(k \times R_{mems}) - h \times (N+k-1) \times \bar{B}} \quad , \quad where \quad k \times R_{mems} > h \times (N+k-1) \times \bar{B}.$$

4.2.2 Striped Cache Management

Theorem 4. The minimum size of per-stream DRAM buffer required to satisfy real-time streaming requirements in a system which streams n streams from an array of k MEMS devices with striped cache management is

$$S_{mems-dram} = \frac{n \times \bar{L}_{mems} \times (k \times R_{mems}) \times \bar{B}}{(k \times R_{mems}) - n \times \bar{B}}, \quad where \quad k \times R_{mems} > n \times \bar{B}. \tag{18}$$

Proof: In striped case, each MEMS device contains $\frac{1}{k}$ of cached streams. All MEMS devices perform operations synchronously. This simplifies streaming scheduler, and we can express MEMS time-cycle as

$$T_{mems} \geq n \times \bar{L}_{mems} + \left(\frac{n}{k} \times \bar{B} \times T_{mems}\right) / R_{mems}. \tag{19}$$

Similarly to Theorem 2, using Equation 19, we can derive Equation 18. □

Corollary 4. Striped cache consisting of k MEMS devices behaves as a single MEMS device with k times larger throughput and unchanged access latency.

For striped case, percentage of cached streams is

$$p = \frac{k \times Size_{mems}}{Size_{disk}}.$$

Using Theorem 4 we find per-stream buffer requirement for access to striped cache as

$$S_{mems-dram} = \frac{h \times N \times \bar{L}_{mems} \times (k \times R_{mems}) \times \bar{B}}{(k \times R_{mems}) - h \times N \times \bar{B}}, \quad where \quad k \times R_{mems} > h \times N \times \bar{B}.$$

5 Experimental Evaluation

This section presents an experimental evaluation of a streaming multimedia system equipped with a MEMS-based storage device. Its performance is compared against a system which does not include such a device in the storage hierarchy. We evaluate the performance of the MEMS device when it is used to buffer streaming data stored on the disk drive, and when it is used to cache popular streams, separately. The evaluation presented in this section is based on the analytical model presented in Section 4. All the experiments in this section are conducted on a hypothetical system which could be developed in the future. The hypothetical system includes a MEMS-based device in its storage hierarchy between the disk drive and DRAM.

Parameter	Atlas 10K	FutureDisk	G3 Mems	DRAM
rpm	10,000	20,000	–	–
maximum bandwidth [MB/s]	55	300	320	10,000
average seek [ms]	4.5	2.8	–	–
full stroke seek [ms]	11.0	7.0	0.45	–
X settle time [ms]	–	–	0.14	–
per-sled capacity [GB]	–	–	10	–
Cost/GB [$]	–	0.2	1	20
Cost/device [$]	100–300	100–300	10	50–200

Table 3: **Performance characteristics of the storage devices in 2002 (today) and in 2007**. The FutureDisk and the reference Atlas 10K III SCSI disk, the G3 MEMS device, and DRAM used in our experiments are based on projections using current trends.

To model disk-drive performance, we use the performance of a hypothetical FutureDisk, obtained by a projection of current disk-drive trends to the year 2007. A current-day Atlas 10K III SCSI disk drive [4] is used as the current-day reference disk for the projection. The performance characteristics of the FutureDisk are presented in Table 3.

To model the performance of the MEMS device, we closely follow one such model proposed by researchers at the Carnegie Mellon University. The work in [6] describes their model extensively. For our experiments, we use the "3rd Generation (G3)" MEMS device model proposed in this study. The G3 model was proposed as the high-end MEMS storage device, one that would typically be used in large server systems. The performance characteristics of the G3 MEMS device relevant to this study are summarized in Table 3.

To model the cost of each of device, DRAM, MEMS, and FutureDisk, we use the two parameters: *cost-per-device* and *cost-per-byte*. These values projected to the year 2007 are presented in Table 3. For disk drives and MEMS storage, a cost-per-byte model is not sufficient to estimate the total cost of the storage device. The total cost of the MEMS device, inclusive of electronics and mechanical components, is greater than the cost of its data recording medium. Although DRAM cost increases almost linearly with its storage capacity, it is available only in modules. However, DRAM can be shared easily, unlike MEMS devices. If the MEMS device throughput is fully utilized, spare storage on the MEMS device is useless. Consequently, we use a cost-per-device model for the MEMS storage and a cost-per-byte model for DRAM.

In our experiments the average bit-rate of streams, $\bar{B}$, is varied in the range 10KB/s to 10MB/s. Since the maximum bandwidth of the FutureDisk, R_{disk}, is 300MB/s, it can support tens of high-definition streams at a few megabytes per second each, more than a hundred compressed MPEG2 (DVD quality) streams at 1MB/s, or a thousand DivX (MPEG4) streams at 100KB/s, or even tens of thousands of MP3 audio or low-resolution video streams at a bit-rate of 10KB/s or less. In order to minimize the misprediction of seek-access characteristics for the MEMS device, we assume that MEMS accesses, L_{mems}, always experience the maximum device latency. In contrast, we use scheduler-determined latency values, $\bar{L}_{disk}$, for disk accesses. The disk IO scheduler uses SCAN (or elevator) scheduling to optimize for disk utilization.

5.1 MEMS Multimedia Buffer

Accessing streaming content through a MEMS buffer (rather than directly accessing the disk drive) can reduce the DRAM requirement of a system. In our experiments, we use at least two G3 MEMS devices for buffering the streaming data, which provide a maximum aggregate MEMS throughput of 640 MB/s. This is greater than twice the disk throughput, as required for buffering purposes. The low cost of these devices (Table 3) compared to the disk drive enables us to use a bank of MEMS devices to buffer IOs for each disk.

To evaluate the performance of the MEMS multimedia buffer, our experiments aim to answer the following questions:

- *By how much does a MEMS buffering device reduce DRAM requirement for various streaming media workloads ?*
- *By how much does a MEMS buffering device reduce the overall system cost ?*
- *How sensitive are each of the above metrics to possible variations in MEMS device characteristics ?*

To answer these questions, we must identify the input and output parameters from our analytical study. These are summarized in Table 2.

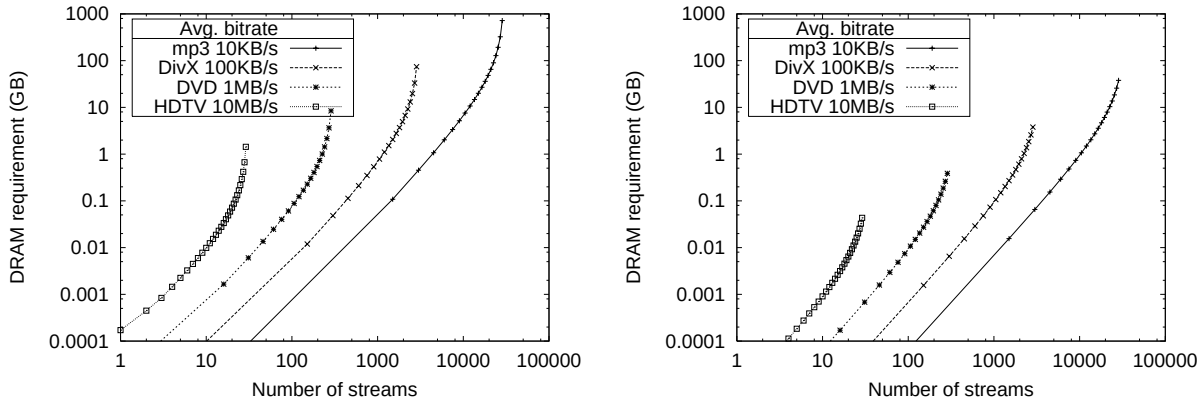
Input: N, $\bar{B}$, L_{disk}, L_{mems}, R_{disk}, R_{mems}.

Output: $S_{disk-dram}$, $S_{disk-mems}$, $S_{mems-dram}$.

Theorems 1 and 2 define the relationship between these parameters. To study the sensitivity of our evaluation to MEMS device characterisitcs, we introduce the *latency ratio*, $\frac{L_{disk}}{L_{mems}}$, as a tunable parameter.
Latency ratio: We define the *latency ratio* as the ratio of the average disk access latency to the maximum MEMS access latency. Throughout the analysis section, we vary the latency ratio within the range 1 to 10. The value for this parameter is around 5 for the FutureDisk (employing elevator scheduling) and the G3 MEMS device presented in Table 3.

Next, we conduct three separate experiments to answer each of our questions. In the first two experiments, we assume that the maximum amount of DRAM and MEMS storage available is unlimited. We also use a cost-per-byte price model for MEMS storage. These relaxations allow us to observe the interplay between all the system parameters. In the third experiment, we perform a case-study using a predicted "off-the-shelf" system which could be developed in the year 2007. The available buffering on this system is limited and based on current trends in server system configurations.

5.1.1 Reduction in DRAM requirement



(a) Without MEMS buffer

(b) With MEMS buffer

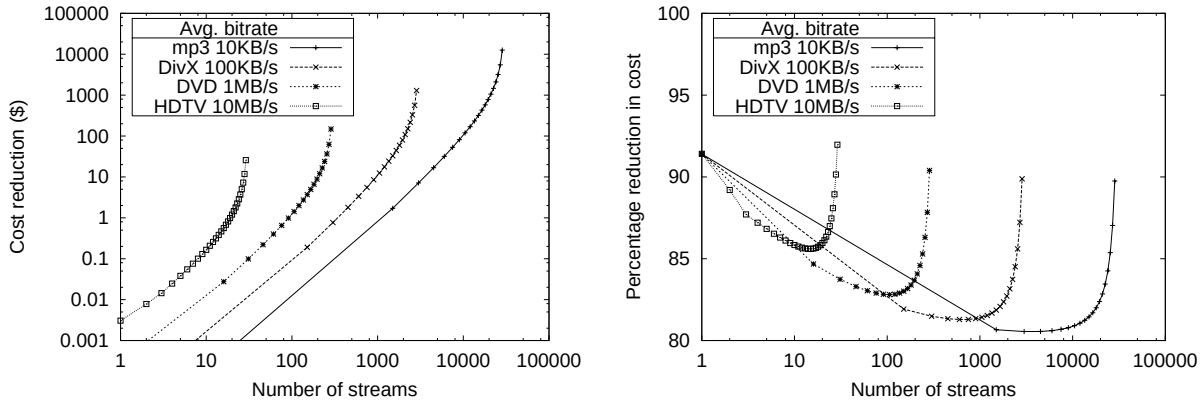
Figure 9: **DRAM requirement for various media types**. Using a MEMS device as disk buffer reduces the DRAM requirement by an order of magnitude.

In Figure 9(a), we vary the number of streams, N, and the average stream bit-rate, $\bar{B}$. We plot the DRAM requirement in GB on the Y-axis. The X and Y axes are drawn to logarithmic scale. The disk drive supports different number of streams for different bit-rate media types. The FutureDisk can support only around 30 HDTV streams, while it can support as many as $30,000$ mp3 streams. Irrespective of the bit-rates, the total buffering requirement increases exponentially with the number of streams (according to Equations 1 and 3). For a fixed system throughput, the buffering requirement is thus much larger for smaller bit-rates than larger ones. In the absence of a MEMS buffer, the DRAM requirement for a fully utilized disk ranges from 1GB for 10MB/s streams to 1TB for 10KB/s streams. In practice, multimedia systems cannot operate at such high utilizations due to the exponential DRAM requirement. If we assume that the available DRAM buffering is limited to 1GB, then the system with a MEMS buffer can support the maximum possible streams when the average stream bit-rate is $1 - 10$MB/s. It can also support support more than twice the number of streams when the average stream bit-rate is in $10 - 100$KB/s range. Looking

at it the other way, the DRAM requirement is reduced by an order of magnitude to support a given system throughput.

5.1.2 Reduction in Cost

Addition of a MEMS buffer reduces DRAM requirement. However, we must we must take the total system cost into account before reaching a conclusion about the benefits. A streaming multimedia system might use multiple disks for storing its streams. Since a MEMS device decreases DRAM requirement, the reduction in the DRAM cost drives the reduction in the total system cost. In this regard, we calculate the cost of buffering, including MEMS and DRAM, per disk. The savings can then be multiplied for each disk in the system.



(a) Cost reduction

(b) Percentage cost reduction

Figure 10: **Reduction in the total buffering cost for various media types**.

Figure 10 presents reduction in the total buffering cost, inclusive of the additional cost of the MEMS buffer. mTo calculate the cost of buffering, we use cost predictions as presented in Table 3. According to the predictions, MEMS buffering is 20 times cheaper than DRAM buffering per-byte.

In Figure 10(a) in spite of the addition of the MEMS buffer, the total cost of buffering is reduced for all media types. Cost savings range from tens of dollars for high bit-rate streams to tens of thousands of dollars for lower bit-rates. These cost savings are almost directly proportional to the DRAM reductions presented in Figure 9, since the fractional cost-addition due to MEMS buffering is negligible in most cases. This curve also points out that using a MEMS buffer for streaming large bit-rates does not cause a significant reduction in the buffering cost. Indeed, for large bit-rates, even DRAM buffering is sufficient to achieve high disk utilization.

Figure 10(b) shows the percentage reduction in the total buffering cost upon addition of the MEMS buffer. We see that the the cost-reduction is approximately between 80% and 90% upon varying the number of streams, N, and their average bit-rates, $\bar{B}$. The cost-reduction is dominated by the cost-ratio of the MEMS and DRAM buffering. The variations in the reduction is due to the relative cost of DRAM, both with and without the MEMS device.

As N is increased, the required throughput from the disk drive increases. The amount of DRAM buffering required to sustain a higher throughput level is greater and the relationship is depicted in Figure 3 for the FutureDisk disk drive and G3 MEMS device. The behaviour of the curves for different media types

is similar, showing that the cost-reduction depend only mainly on the total throughput desired from the system and not so much on the individual values of $\bar{B}$ and N. The slight variations in the absolute values of these curves are due to the variations in disk latency (depending on N) due to the elevator scheduling policy employed. For larger values of N, the average latency is lesser.

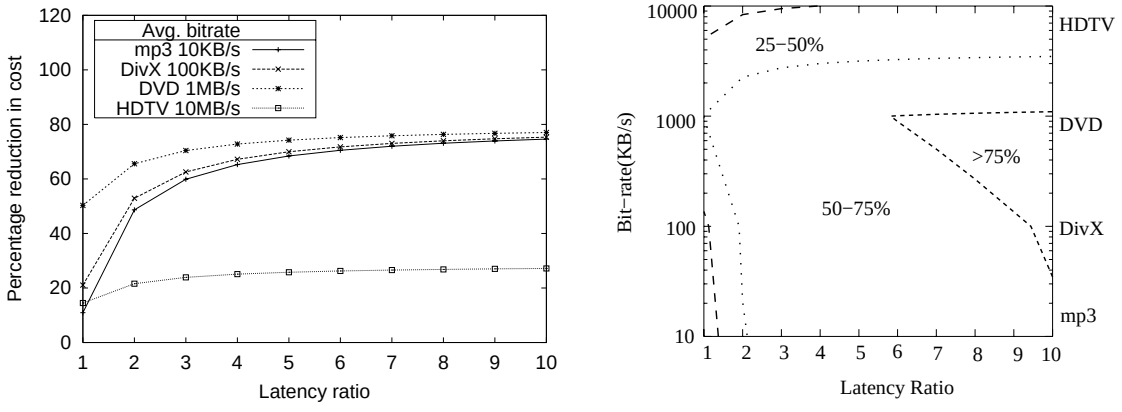
5.1.3 Case Study

We now present a case-study of an off-the-shelf system which could be developed in the future. We compare a system without MEMS buffering with one that uses a MEMS buffer. Using this off-the-shelf system, we determine the sensitivity of cost-reductions to unpredictable trend changes in device characteristics.

The performance of any MEMS-based disk buffer depends on its physical access characteristics. Earlier in the section, we defined a sensitivity parameter, *latency-ratio*. This parameter has a predicted value of 5 for the FutureDisk disk drive and G3 MEMS device used in earlier experiments. However, this value is sensitive to the trends in future MEMS and disk-drive characteristics. In this section, we present how the cost-savings presented earlier are sensitive to future trends in the access characteristics of these devices.

Varying the latency-ratio ($\frac{L_{disk}}{L_{mems}}$)

In our earlier experiments, we assumed that the system can use an unlimited amount of DRAM and/or MEMS storage and using a cost-per-byte model for MEMS buffering. However, a system with terabytes of DRAM and/or buffering is very expensive and practically unrealistic. To perform our next set of experiments, we restrict the system configuration to that of off-the-shelf system configurations projected to the year 2007. These configurations impose restrictions that the disk drive does not operate at utilizations where the buffering requirement is impractically large. For a system that supports different media types, the moderate utilization range depends on the average stream bit-rate, $\bar{B}$ and the economically viable DRAM buffering, M_{dram}, available. Therefore, to make a realistic evaluation, we choose a fixed M_{dram}, and obtain N for different average stream bit-rates for a system without the MEMS buffer. We then calculate the savings in DRAM obtained on addition of a MEMS buffer to support the same number (N) of streams.



(a) Cost-reduction curves

(b) Cost-reduction regions

Figure 11: **Percentage cost reduction**.

In the following experiment, the maximum available DRAM is fixed at 5GB, to obtain N in the absence of a MEMS buffer. The size of the DRAM is chosen based on trend projection for server memory to the year 2007. Cost predictions for the year 2007 (see Table 3) indicate that a 5GB DRAM buffer would sell for $100. The MEMS buffering cost is calculated using a cost-per-device model. Table 3 indicates that a 10GB MEMS device would sell for $10. We calculate the reduction in DRAM requirement with the addition of two MEMS devices to obtain a 20GB MEMS buffer. Factoring the cost of the MEMS device, we plot the percentage cost-reduction curve in Figure 11.

Figure 11(a) depicts the percentage reduction in the buffering cost for different average stream bit-rates when the latency ratio is varied. As the latency-ratio is increased, the access latency of the MEMS device improves relative to the disk access latency. Consequently, the MEMS device can deliver higher throughputs with lesser buffering. As a result, the reduction in buffering cost increases. The maximum possible reduction in buffering cost is bound by the fixed cost of MEMS buffering using 2 MEMS devices. When the average bit-rate of the streams in the system is small, the amount of DRAM buffering required even in the presence of the MEMS buffer, is relatively large.

Another interesting observation is that the cost-reduction trend is is reversed at large bit-rates, 10MB/s in Figure 11(a). When the average bit-rate is increased in the lower range, the performance of both the disk drive and the MEMS device improves. However, when the average stream bit-rate is increased, the differential improvement in throughput for the MEMS device is more rapid due to its faster access characteristics (See Figure 3). The savings in DRAM and hence buffering cost also increase. However, beyond a certain average bit-rate (1MB/s in this case), the savings in DRAM begin to decrease. In this region, the DRAM requirement in the absence of the MEMS buffer is small and the 5GB DRAM buffering available is under-utlilized even without the MEMS buffer. In the absence of a MEMS buffer, the DRAM requirement for the 10MB/s bit-rate range is approximately 1.5GB, thereby limiting the cost-reduction to only 30%.

In Figure 11(b), contour lines on the XY-plane depict the regions in which the percentage reduction in the total buffering cost are 25%, 50%, and 75%. This implies that the MEMS buffer is cost-effective almost over the entire parameter space. In a majority of the parameter space, there is atleast a 50% to 75% reduction in the total buffering cost. The cost-reductions observed here are much lesser compared to those presented in Figure 10. This is due to the cost-per-device model assumption for the MEMS buffer. The MEMS device used in the current experiment costs 20 and thereby restricts the cost-savings to less than 80%. Also, even if all of the MEMS buffer is not used, the remaining storage available cannot be used due to bandwidth restrictions at the MEMS device. In contrast, Figure 10 used a cost-per-byte model and an unlimited amount of MEMS buffer.

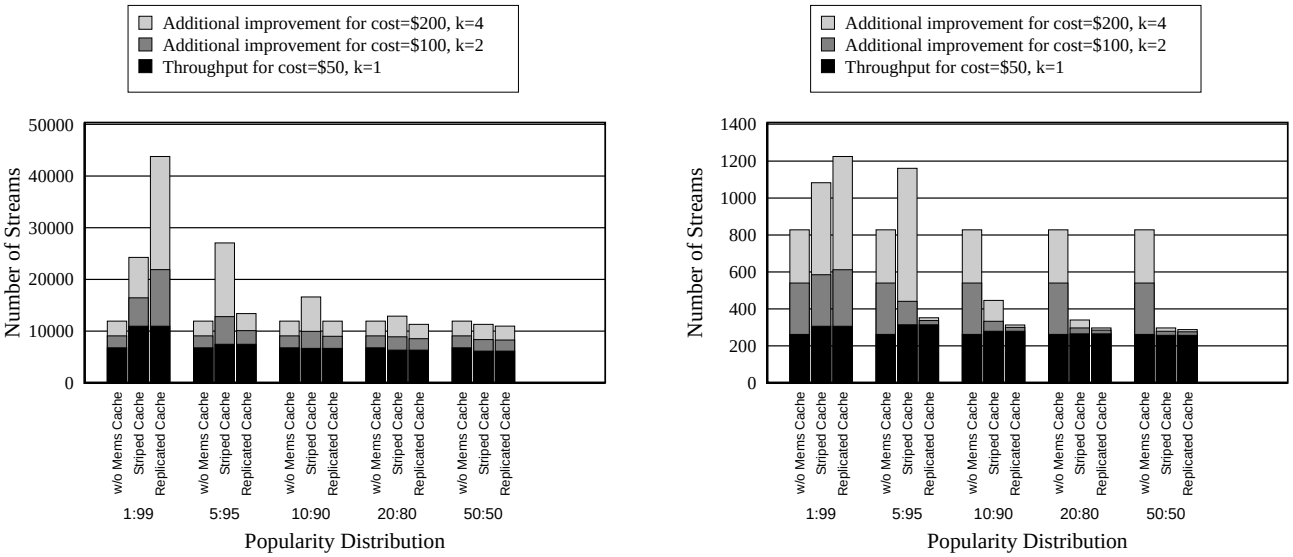
5.2 MEMS Multimedia Cache

When data stored on the disk drive has a non-uniform access probability, then using the MEMS device as a cache for popular streams can improve the performance of a multimedia server as analyzed in Section 4. However, the performance of the MEMS cache depends on three factors: the amount of MEMS cache available, the popularity distribution of streams, and the average stream bit-rate. To measure the performance of the MEMS cache, we calculate the improvement in the server throughput in terms of the number of additional streams serviced using a MEMS cache by fixing the total cost of the system. The additional cost due to the MEMS cache is reflected by a reduced amount of available DRAM buffering. We evaluate the performance of the MEMS cache using the two cache-management policies described in Section 3: *striped* and *replicated*. To evaluate the performance of the MEMS cache, our experiments aim to answer the following questions:

- *What is the improvement in the server throughput for various popularity distributions of streams ?*

- *What is the improvement in the server throughput when the total cost is flexible ?*
- *How does the improvement in server throughput depend on the average stream bit-rate ?*
- *What is the optimal size (k) of the MEMS cache for a given server setting ?*

The number of streams serviced from the MEMS device depends on the number of cache hits and follows Equation 13. To determine the cache hits, we use the size of the MEMS device relative to the size of the disk drive. The FutureDisk has a storage capacity of a terabyte, implying that each 10GB MEMS device can cache only 1% of the content. In our experiment, we assume that if a stream is found on the cache, it will be serviced only from the cache[5].



(a) Average bit-rate 10KB/s

(b) Average bit-rate 1MB/s

Figure 12: **MEMS cache performance**. The performance of the MEMS cache using two cache-management policies: *striped* and *replicated*, are compared against that of a system witout a MEMS cache.

To answer our questions, we conduct the following experiment. We fix the cost of the system and vary the popularity distribution to determine the server throughput with and without a MEMS cache. In order to fix the total cost, we use the cost predictions for MEMS and DRAM as presented in Table 3. Each MEMS caching device added reduces the amount of DRAM buffering available by 500MB to keep the cost invariant. The popularity distribution follows X:Y, where X% of the streams are accessed Y% of the time. We also assume that all the X% of the streams are equally popular. Similarly, we assume that the remaining (100-X)% of the streams are equally unpopular. We conduct the same experiment for three different values of the total buffering cost: \$50, \$100, and \$200. The number of MEMS devices, k, used for caching for these buffer costs are chosen as 1, 2, and 4 respectively. Later, we examine the effect of varying k for each value of the buffering cost. The entire experiment is performed for two different average stream bit-rates: 10KB/s, and 10MB/s. Figure 12 provides the basis for answering our three questions.

[5]It can be shown that the performance of the cache can be further enhanced by off-loading the servicing of some popular streams to the disk drive if the disk is under-utilized. We leave this study for future work.

5.2.1 Effect of popularity distribution

Figure 12(a) compares the performance of the two cache-management policies: *striped* and *replicated*, against a system without a MEMS cache. The X-axis represents the popularity distribution and the Y-axis represents the server throughput in terms of the number of streams serviced. The average stream bit-rate is fixed at 10KB/s.

The performance of the system without a MEMS cache is, of course, independent of the popularity distribution. A uniform popularity distribution is denoted by the point 50 : 50 along the X-axis. When the number of MEMS devices is one, the performance of the replicated cache is equal to the performance of the striped cache, since they are equivalent. For skewed popularity distributions of 1 : 99, 5 : 95, and 10 : 90, both cache management policies outperform the system without the MEMS cache. The MEMS cache can support more streams than the disk drive due to its lower access latencies and higher throughput. However, when the popularity distribution is towards the uniform side the MEMS caching is not cost-effective. The MEMS cache is able to store only a small fraction of the popular movies, thereby failing to offset its cost.

We can also compare the relative performance of the two cache-management policies. When the popularity distribution is greatly skewed (1 : 99), the replicated cache-management policy supports the maximum number of streams. Since all the popular streams are available on the cache using either policy, replication outperforms striping by offering much lower average access latencies to the MEMS cache. This is not the case for more uniform popularity distributions, when striping can cache more popular content than replication.

5.2.2 Effect of varying the total cost

If the cost of the system is flexible, a system designer must know the size of the MEMS cache and DRAM buffer which provides the best server performance. In this regard, we vary the amount of MEMS cache and DRAM buffer available per disk to determine the effect on server throughput. Caching improves system performance when the number of MEMS devices is large enough to cache a majority of the popular movies. More MEMS devices can be used for caching when the available budget for buffering and caching is large. The greater the available budget, the bigger is the range of popularity distributions where the MEMS cache is cost-effective. For instance, in Figure 12(a), with a \$50 budget, the MEMS cache (using one MEMS device) is cost-effective only in the popularity range 1 : 99 to 5 : 95. With a higher budget (\$200), the MEMS cache with 4 devices is cost-effective over a greater popularity range 1 : 99 to 20 : 80. In a later experiment, we vary the number of MEMS devices for a fixed budget to determine the optimum number of MEMS devices for a given cost configuration.

5.2.3 Effect of varying the average stream bit-rate

The performance improvement due to the MEMS cache also depends on the average bit-rate of the streams served. For larger bit-rates, the disk-drive is able to deliver high throughput levels. Although the addition of the MEMS cache increases the total server throughput, the relative increase is less when compared with throughput improvements for smaller average bit-rates. Figures 12(a) and (b) show this behavior. When the average bit-rate is small, the MEMS device reduces the per-stream DRAM requirement drastically and is able to support a far greater number of streams.

5.2.4 Effect of varying the number of MEMS devices

Figure 13 shows the effect of varying the size of the MEMS cache, k, on the improvement in the server throughput. Striped cache-management policy is used for this experiment. The total cost of buffering

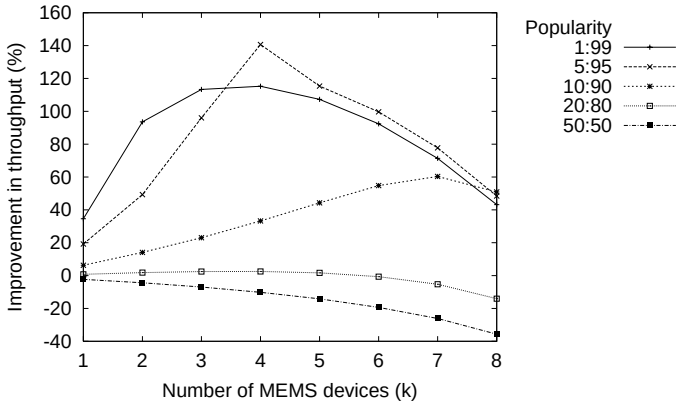


Figure 13: **Varying the size of the MEMS cache**. The X-axis represents the number of MEMS devices in the cache. The system serves 100KB/s streams and the striped management policy is used for the MEMS cache.

and caching with and without the MEMS cache is fixed at $100 and the system serves an average bit-rate of 100KB/s. Each MEMS device can cache 1% of the content. As k is increased, the throughput of the MEMS cache increases. However, to keep the total cost invariant, the available DRAM buffer size decreases. Hence, for each popularity distribution, there exists a unique optimal size of the MEMS bank.

When the popularity distribution is uniform, represented by $50:50$, the MEMS cache always degrades the server performance. For skewed popularity distributions ($1:99$, $5:95$, and $10:90$), the MEMS cache improves the server throughput by as much as 140%.

5.3 Results Summary

Using MEMS buffer as a disk buffer or cache can both improve system performance. However, the performance improvement is not consistent over all system configurations. We summarize our findings as follows:

- Using a MEMS device as a disk buffer can improve the system performance when the server load requires a large amount of buffering. This is true when the average bit-rate of the streams served is small. The performance improvement due to a MEMS-based disk buffer is as much as 75% when the average bit-rate is less than or equal to 1MB/s. For larger stream bit-rates, the MEMS device simply does not prove cost-effective.

- If content has a non-uniform popularity distribution, using a MEMS device as a disk cache can improve the throughput of streaming servers. The MEMS cache provides additional streaming bandwidth at the server. The improvement is greater for smaller bit-rates. However, there is degradation in the server throughput if the content does not have a sufficiently skewed popularity distribution.

- When the popularity distribution is not sufficiently skewed (for e.g., $20:80$ to $50:50$ in Figure 12), it is better to use the MEMS device as a buffer rather than a cache. In this range, caching does not improve server throughput, while buffering can reduce the system cost by as much as 75%. The range of popularity distributions where the cache is cost-effective depends on the average stream bit-rate.

6 Related Work

Multimedia workloads fall into two main categories: (1) *best-effort*, and (2) *real-time*. Textual and image data require a *best-effort* service. In most cases, retrieval of these data require prompt service and must be starvation-free. Continuous data forms like audio and video fall into the *real-time* category, imposing real-time requirements on data retrieval. In order to maintain stream continuity, these data are usually retrieved in rounds or cycles in a timely fashion. Further, in each round, data must be retrieved in a timely fashion. For instance, to ensure jitter-free playback, video data must be retrieved so that the display buffer is always non-empty.

The *best-effort* class of multimedia data usually display spatial and/or temporal locality attributes. Traditional caching policies [] are well suited for such applications. The work of [6] shows that including a MEMS-based cache in the storage hierarchy can improve I/O response time by up to 3.5X for the best-effort class of multimedia data. The focus of this work is to examine if such an addition can also improve the performance of the other class of multimedia data: the *real-time* class.

7 Conclusions and Future Work

We proposed and investigated the possibility of using MEMS devices for buffering and caching streaming multimedia content. Although this is a speculative study considering that MEMS devices are still years away from commercial mass production, we believe that this study is a first step towards using these devices for real-time applications like streaming multimedia. We hope that the result of this work can provide guidelines for designing next generation multimedia servers.

Our work can be further explored in at least two directions. We realize that the MEMS device could be simultaneously used for buffering as well as caching popular streams. For instance, when the popularity distribution is not skewed sufficiently to highly utilize the MEMS device as a cache, part of the MEMS storage (storing the popular streams currently not being serviced) can be used as a buffering medium to increase the disk throughput. This work can also be extended to include intelligent placement policies for data on the MEMS device so as to improve the access characteristics of these devices for multimedia data.

References

[1] E. Chang and H. Garcia-Molina. Effective memory use in a media server. *Proceedings of the 23rd VLDB Conference*, pages 496–505, August 1997.

[2] S. J. Daigle and J. K. Strosnider. Disk scheduling for multimedia data streams. *Proceedings of the IS&T/SPIE*, February 1994.

[3] J. Grin, S. Schlosser, G. Ganger, and D. Nagle. Operating systems management of mems-based storage devices. *Proceedings of the 4th Symposium on Operating Systems Design and Implementation*, 2000.

[4] Maxtor Corporation. Atlas 10KIII-U320 product datasheet. *http://www.maxtor.com/Quantum/rios/data/sources/binary/Atlas10kIIIu320_DataSheet.pdf*, June 2002.

[5] P. V. Rangan, H. M. Vin, and S. Ramanathan. Designing and on-demand multimedia service. *IEEE Communications Magazine*, 30(7):56–65, July 1992.

[6] S. W. Schlosser, J. L. Griffin, D. Nagle, and G. R. Ganger. Designing computer systems with MEMS-based storage. *Architectural Support for Programming Languages and Operating Systems*, pages 1–12, 2000.

[7] D. A. Thompson and J. S. Best. The future of magnetic data storage technology. *IBM Journal of Research and Development*, 44(3), May 2000.

[8] P. Vettiger, M. Despont, U. Drechsler, U. Durig, W. Haberle, M. I. Lutwyche, H. E. Rothuizen, R. Stutz, R. Widmer, and G. K. Binning. The "Millipede" - more than one thousand tips for future afm data storage. *IBM Journal of Research and Development*, 44(3):323–340, 2000.